

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ

FACULTAD DE CIENCIAS E INGENIERÍA

PROGRAMACIÓN 3 6ta. práctica (tipo a) (Primer Semestre 2026)

Indicaciones Generales:

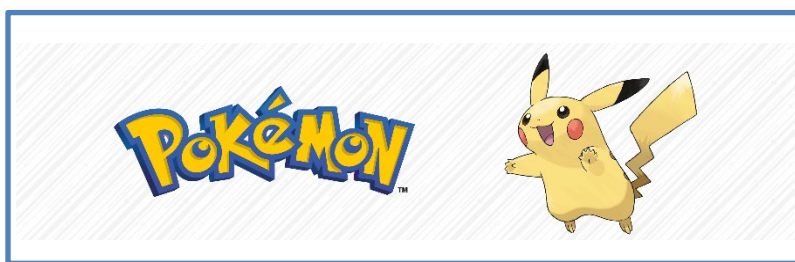
- Tiempo: 1h 50 minutos
- Se les recuerda que, de acuerdo al reglamento disciplinario de nuestra institución, constituye una falta grave copiar del trabajo realizado por otro estudiante o cometer plagio para el desarrollo de esta práctica.
- Para el desarrollo de toda la práctica debe utilizar el sistema operativo **Windows**, así como **Visual Studio 2026**.
- Está permitido el uso de apuntes de clase, diapositivas, ejercicios de clase y código fuente.
(Debe descargarlos antes de iniciar con la solución del enunciado)
- Está permitido el uso de Internet (únicamente para consultar páginas oficiales de Microsoft y Oracle).
No obstante, está prohibida toda forma de comunicación con otros estudiantes o terceros.

Puntaje total: 20 puntos

Parte Práctica

Pregunta 1 (20 puntos)

En un mundo fantástico habitan criaturas conocidas como **Pokémon**, cada una con características únicas que permiten diferenciarlas, tales como su nombre, altura, peso, tipo y estado evolutivo. Estas criaturas forman parte de un ecosistema diverso, donde su clasificación resulta fundamental para su estudio y comprensión.



Para efectos del presente ejercicio, se considerará que cada Pokémon pertenece a un único tipo, con el objetivo de simplificar el modelo de datos.

En sistemas de información, es común que los datos se almacenen inicialmente en estructuras desnormalizadas, lo cual puede generar redundancia e inconsistencias. Por ello, es necesario aplicar procesos de normalización que permitan organizar adecuadamente la información.

1.1. Estructura de datos de entrada

Se cuenta con una base de datos MySQL que almacena información de Pokémon en una tabla desnormalizada denominada **pokemon_tipo_raw**.

```
CREATE TABLE pokemon_tipo_raw (
  id_raw INT AUTO_INCREMENT PRIMARY KEY,
  nombre_pokemon VARCHAR(80) NOT NULL,
  altura DECIMAL(5,2),
  peso DECIMAL(5,2),
  estado_evolutivo ENUM('BASICO', 'INTERMEDIO', 'FINAL') NOT NULL,
  nombre_tipo VARCHAR(50) NOT NULL,
  descripcion_pokemon VARCHAR(255)
);
```

En esta tabla, cada registro contiene información del **Pokémon** junto con su tipo, lo que genera redundancia en los datos del tipo.

1.1.1 Ejemplo de registros

```
INSERT INTO pokemon_tipo_raw
(nombre_pokemon, altura, peso, estado_evolutivo, nombre_tipo, descripcion_pokemon)
VALUES
('BULBASAUR', 0.70, 6.9, 'BASICO', 'PLANTA', 'TRAS NACER, CRECE ALIMENTÁNDOSE DE LOS NUTRIENTES DE LA SEMILLA QUE LLEVA EN EL LOMO.'),
('IVYSAUR', 1.00, 13.0, 'INTERMEDIO', 'PLANTA', 'CUANDO EL BULBO DE SU ESPALDA CRECE, PARECE PERDER MOVILIDAD.'),
('VENUSAUR', 2.00, 100.0, 'FINAL', 'PLANTA', 'LA PLANTA FLORECE AL ABSORBER ENERGÍA SOLAR.'),
('CHARMANDER', 0.60, 8.5, 'BASICO', 'FUEGO', 'LA LLAMA DE SU COLA INDICA SU ESTADO DE SALUD.'),
('PIKACHU', 0.40, 6.0, 'BASICO', 'ELECTRICO', 'ESTE INTELIGENTE POKÉMON TUESTA LAS DURAS BAYAS CON ELECTRICIDAD PARA HACERLAS MÁS COMESTIBLES.');
```

1.2. Modelo de Datos Objetivo (NORMALIZADO)

Se desea transformar la información hacia el siguiente modelo:

```
CREATE TABLE tipo_pokemon (
    id_tipo INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE pokemon (
    id_pokemon INT AUTO_INCREMENT PRIMARY KEY,
    fid_tipo INT NOT NULL,
    nombre VARCHAR(80) NOT NULL,
    altura DECIMAL(5,2),
    peso DECIMAL(5,2),
    estado_evolutivo ENUM('BASICO', 'INTERMEDIO', 'FINAL') NOT NULL,
    descripcion VARCHAR(255),
    FOREIGN KEY (fid_tipo) REFERENCES tipo_pokemon(id_tipo)
);
```

1.3. Objetivo

Desarrolle una aplicación en C# que permita:

- Leer todos los registros de la tabla **pokemon_tipo_raw** y almacenarlos en una estructura intermedia (por ejemplo, una lista de DTO).
- Transformar los datos leídos hacia el modelo normalizado propuesto.
- Insertar los tipos en la tabla **tipo_pokemon**, evitando registros duplicados.
- Insertar los Pokémon en la tabla **pokemon**, asociando cada registro con su tipo correspondiente mediante la clave foránea respectiva.
- Manejar el atributo **estado_evolutivo** como un tipo enumerado en la aplicación.
- Implementar la solución empleando programación por capas, de acuerdo con lo trabajado en clase. Para ello, se deberán considerar los proyectos de Dominio, Persistencia, Negocio y DBManager, así como un proyecto de aplicación de consola que actúe como punto de entrada para la ejecución del proceso.
- No será necesario implementar transacciones, debido al alcance académico del ejercicio.

Para el desarrollo del ejercicio, se están proporcionando scripts SQL que incluyen la creación de la tabla desnormalizada, las tablas normalizadas, la carga de datos y los procedimientos almacenados necesarios en caso desee utilizarlos. Este script deberá ejecutarlo en su instancia de MySQL en AWS y desarrollar la solución en C# conectándose a dicha base de datos. **Suba en un archivo de texto las credenciales de acceso a su instancia de MySQL.**

Utilice la solución inicial de Visual Studio que se le está proporcionando que ya tiene definida la capa del dominio considerando la clase DTO.

1.4. Consideraciones

- Para efectos del presente ejercicio, cada Pokémon pertenece a un único tipo.
- No existen registros duplicados de Pokémon en la tabla desnormalizada.
- El campo **nombre_tipo** puede repetirse en la tabla desnormalizada.
- Todos los datos se encuentran almacenados en mayúsculas.
- El atributo **estado_evolutivo** deberá ser tratado como un tipo enumerado tanto en la base de datos como en la aplicación.
- Se espera una correcta implementación de la solución bajo un enfoque de programación por capas.
- El presente ejercicio representa una versión simplificada de un proceso **ETL (Extract, Transform, Load)**:
 - **Extract:** lectura de información desde la tabla desnormalizada `pokemon_tipo_raw`.
 - **Transform:** transformación de los datos hacia el modelo normalizado y conversión de atributos enumerados.
 - **Load:** inserción de la información en las tablas `tipo_pokemon` y `pokemon`.
- A continuación, se muestra un ejemplo referencial de lectura de un atributo enumerado desde un `DataReader`:

```
cliente.Categoria = (Categoria) Enum.Parse(typeof(Categoria), lector.GetString("categoria"));
```

donde `Categoria` es un tipo enumerado.

- El proceso de migración deberá realizarse a través de una clase de la capa de negocio llamada `MigratorBOImpl` o `MigratorBL`, implementando un método `run()` responsable de ejecutar el proceso de normalización de datos.

1.5. Lógica de Referencia (Tip para la solución)

A continuación, se presenta una lógica general de referencia para la resolución del ejercicio:

1. Leer todos los registros de la tabla **pokemon_tipo_raw** y almacenarlos en una estructura intermedia (por ejemplo, una lista de DTO).
 2. Recorrer los registros obtenidos y, por cada uno:
 - a. Construir el objeto **TipoPokemon** y verificar si el tipo (**nombre_tipo**) ya existe en la tabla **tipo_pokemon**.
 - En caso de existir, obtener su identificador.
 - En caso contrario, insertar el tipo y obtener el identificador generado.
 - b. Construir el objeto **Pokemon**.
 - c. Insertar el Pokémon en la tabla **pokemon**, asociándolo con el identificador del tipo correspondiente.
- Repetir el proceso hasta completar todos los registros.

1.6. Nota de propiedad intelectual

Pokémon es una marca registrada de **The Pokémon Company**. Los nombres, descripciones y referencias utilizadas en este ejercicio se emplean exclusivamente con fines académicos y educativos.

Profesores del Curso.

San Miguel, 06 de mayo del 2026